

UNIT III: CONTROL UNIT

Control Unit: Instruction types, formats, instruction cycles and sub cycles (fetch and execute etc), micro operations, execution of a complete instruction. Program Control, Reduced Instruction Set Computer, Pipelining. Hardwire and micro programmed control: micro programme sequencing, concept of horizontal and vertical microprogramming.

◇ 1. Introduction to Control Unit

The **Control Unit (CU)** is the part of the **CPU (Central Processing Unit)** that directs the operation of the processor.

It **tells the memory, ALU, and input/output devices how to respond** to the instructions sent to the processor.

► Main Function:

- It **fetches, decodes**, and **executes** instructions.
- It **controls data flow** between CPU, memory, and I/O devices.
- It **generates control signals** to manage timing and execution.

🧩 Components:

1. **Instruction Register (IR)** – holds the current instruction.
2. **Decoder** – interprets the opcode part of the instruction.
3. **Control Logic Gates** – generate control signals.
4. **Timing Signals** – synchronize all operations.

◇ 2. Instruction Types

Different instructions perform different operations.

Type	Description	Example
Data Transfer	Move data between registers or between memory and register	MOV A, B
Arithmetic	Perform operations like add, subtract	ADD A, B
Logical	Perform AND, OR, NOT operations	AND A, B
Branch (Control)	Change the sequence of execution	JMP LABEL
Input/Output	Handle I/O devices	IN PORT1, OUT PORT2

◇ 3. Instruction Format

An **instruction format** defines how bits in an instruction word are divided into fields.

Common Fields:

Field	Description
Opcode	Operation to perform (e.g., ADD, MOV)
Source Operand	Data to operate on
Destination Operand	Where to store result
Address Field	Memory address for operand

Example:

Suppose instruction = ADD R1, R2

- **Opcode** = ADD
- **Source** = R2
- **Destination** = R1

◇ 4. Instruction Cycle

Each instruction executes in several small steps called **cycles**.

► Basic Steps:

1. Fetch Cycle

→ Get instruction from memory.

Example: $MAR \leftarrow PC$, $MBR \leftarrow Memory[MAR]$, $IR \leftarrow MBR$, $PC \leftarrow PC + 1$

2. Decode Cycle

→ Decode the instruction to find the operation and operands.

Example: $Decoder \leftarrow IR[Opcode]$

3. Execute Cycle

→ Perform the operation (ALU, Memory, or Branch).

Example: $R1 \leftarrow R1 + R2$

4. Store/Write Back Cycle

→ Store the result into memory or register.

◇ 5. Sub-Cycles (Fetch and Execute)

Cycle	Description	Example
Fetch	Fetch instruction from memory	$IR \leftarrow Memory[PC]$
Execute	Execute instruction	$ACC \leftarrow ACC + R1$

◇ 6. Micro-Operations

Each instruction is executed by a sequence of **micro-operations**.

Micro-operations = elementary operations performed on data stored in registers.

Types:

Type	Example
Register Transfer	$R1 \leftarrow R2$
Arithmetic	$R1 \leftarrow R2 + R3$
Logic	$R1 \leftarrow R2 \text{ AND } R3$
Shift	$R1 \leftarrow \text{SHR } R1$

Example:

Instruction: ADD R1, R2

Micro-operations:

1. T1: $MAR \leftarrow PC$
2. T2: $MBR \leftarrow \text{Memory}[MAR]$
3. T3: $IR \leftarrow MBR$
4. T4: Decode IR
5. T5: $R1 \leftarrow R1 + R2$

◇ 7. Execution of a Complete Instruction

The CU performs:

1. **Fetch** – instruction from memory
2. **Decode** – opcode and operands
3. **Execute** – perform the operation
4. **Store** – write back result

Each step = series of micro-operations controlled by timing signals (T0, T1, T2, etc.)

◇ 8. Program Control

It deals with **how the sequence of execution** of instructions is managed.

► **Control Instructions:**

Type	Description	Example
Branch	Jump to another address	JMP 2050
Conditional Branch	Jump only if condition true	JZ LABEL
Subroutine Call	Jump to a function and return	CALL, RET
Interrupt	External event causes CPU to pause	Keyboard interrupt

◇ 9. RISC (Reduced Instruction Set Computer)

RISC simplifies the processor design by using **simple, fast instructions**.

Feature	RISC	CISC
Instruction Set	Small, simple	Large, complex
Execution Time	One cycle per instruction	Many cycles
Addressing Modes	Few	Many
Control	Hardwired	Microprogrammed
Example	ARM, MIPS	Intel x86

⚙️ Key Idea:

Perform more work through **software (compiler)** than hardware.

◇ 10. Pipelining

Pipelining allows multiple instructions to be executed **in overlapping phases**.

► Stages:

1. Fetch
2. Decode
3. Execute
4. Memory Access
5. Write Back

→ Like a **factory assembly line**: while one instruction is executing, another is being fetched.

⚡ Example:

Clock Cycle	Instruction 1	Instruction 2	Instruction 3
1	Fetch	—	—
2	Decode	Fetch	—
3	Execute	Decode	Fetch

✅ **Result:** Higher throughput, faster CPU.

◇ 11. Hardwired Control vs Microprogrammed Control

Basis	Hardwired Control	Microprogrammed Control
Design	Uses fixed logic circuits	Uses microinstructions stored in control memory
Speed	Very fast	Slightly slower

Flexibility	Difficult to modify	Easy to modify (change microprogram)
Cost	Cheaper	More expensive (due to control memory)
Example	RISC processors	CISC processors

◇ 12. Microprogram Sequencing

Microprogramming = control signals are generated using a **microprogram** stored in control memory.

Steps:

1. Each machine instruction → microprogram
2. Microprogram = sequence of **microinstructions**
3. Each microinstruction activates control signals for one clock cycle

◇ 13. Horizontal vs Vertical Microprogramming

Feature	Horizontal	Vertical
Control Signals	Each bit directly controls a signal	Encoded form (needs decoding)
Speed	Faster	Slower
Memory Requirement	More	Less
Flexibility	High	Limited
Complexity	High	Simple

Example:

- **Horizontal:** 1 bit = 1 control line
- **Vertical:** 1 field = group of encoded signals

1. Hardwired Control Unit vs Microprogrammed Control Unit

Feature	Hardwired Control Unit	Microprogrammed Control Unit
Definition	Control signals are generated by fixed combinational logic circuits.	Control signals are generated by executing microinstructions stored in control memory.
Speed	Very fast (no microinstruction decoding).	Slower (each instruction broken into microinstructions).

Flexibility	Difficult to modify.	Easy to modify by changing microprogram.
Complexity	Complex wiring and logic.	Simpler design using control memory.
Cost	Less costly.	More costly due to control memory.
Used in	RISC processors.	CISC processors.
Example	ARM, MIPS.	Intel x86.

? 2. RISC vs CISC Processors

Feature	RISC (Reduced Instruction Set Computer)	CISC (Complex Instruction Set Computer)
Instruction Set	Small and simple.	Large and complex.
Execution Time	Each instruction executes in one clock cycle.	Takes multiple cycles per instruction.
Addressing Modes	Few.	Many.
Control Unit	Hardwired.	Microprogrammed.
Registers	Many general-purpose registers.	Few general-purpose registers.
Code Size	Large (more instructions).	Smaller (complex instructions).
Performance	Faster due to pipelining.	Slower comparatively.
Example	ARM, MIPS.	Intel x86, VAX.

3. Horizontal vs Vertical Microprogramming

Feature	Horizontal Microprogramming	Vertical Microprogramming
Control Signals Representation	Each bit controls a separate signal (unencoded).	Encoded fields represent control signals (decoded before use).
Speed	Faster (no decoding required).	Slower (needs decoding).
Memory Usage	High (wide control word).	Low (narrow control word).
Flexibility	Highly flexible.	Less flexible.
Complexity	Complex design.	Simpler design.
Example Use	High-speed processors.	Microcontrollers, simpler CPUs.

4. Fetch Cycle vs Execute Cycle

Basis	Fetch Cycle	Execute Cycle
Purpose	Get instruction from memory.	Perform operation defined by instruction.
Involves	Program Counter (PC), Memory Address Register (MAR), Memory Buffer Register (MBR).	ALU, registers, and control signals.
Micro-Operations	$MAR \leftarrow PC$, $MBR \leftarrow \text{Memory}[MAR]$, $IR \leftarrow MBR$, $PC \leftarrow PC + 1$.	Depends on instruction type (ADD, MOV, etc.).
Output	Instruction loaded into IR.	Result stored in register or memory.
Duration	Same for all instructions.	Varies for each instruction.

5. Micro-Operations vs Macro-Instructions

Feature	Micro-Operation	Macro-Instruction (Machine Instruction)
Definition	Elementary operation performed on data in registers.	Instruction given by programmer to CPU.
Level	Low-level, hardware control.	High-level, software visible.
Example	$R1 \leftarrow R1 + R2$.	ADD R1, R2.
Execution Time	1 clock pulse.	Many clock pulses (sequence of micro-ops).
Controlled By	Control signals.	Control Unit.

6. Pipelined vs Non-Pipelined Execution

Feature	Pipelined Execution	Non-Pipelined Execution
Concept	Multiple instructions executed in overlapping stages.	One instruction executed completely before the next starts.
Speed	Higher throughput (more instructions per time).	Slower.
Hardware Cost	More hardware for pipeline stages.	Less hardware.

Dependency Handling	Needs hazard detection & control.	No hazards.
Used In	RISC processors.	Simple microcontrollers.

7. Control Transfer Instructions vs Data Transfer Instructions

Basis	Control Transfer	Data Transfer
Purpose	Change sequence of execution.	Move data between registers/memory.
Affects PC	Yes (changes program counter).	No.
Examples	JMP, CALL, RET, JZ.	MOV, LOAD, STORE.